

Documento de Especificação de Requisitos

Este documento consolida todos os requisitos do sistema, divididos em Requisitos Funcionais, Requisitos de Qualidade e Requisitos Técnicos.

1. Requisitos Funcionais

O sistema deve disponibilizar funcionalidades para cadastro, consulta, análise de dados e apoio à tomada de decisão, contemplando clientes, produtos, pedidos, indicadores comerciais e classificação estratégica de clientes.

RF01 - Cadastro de Clientes

O sistema deve permitir o cadastro de clientes contendo, no mínimo, as seguintes informações:

- ID
- Nome
- E-mail
- Cidade
- Estado
- País

O cadastro deve possuir validações obrigatórias, garantindo que os dados essenciais sejam informados corretamente. O campo de e-mail deve ser obrigatório, não podendo estar vazio, e deve seguir um formato válido.

Exemplo de validação esperada:

- **E-mail válido:** cliente@email.com
- **E-mail inválido:** clienteemail.com

Essa funcionalidade serve como base para o relacionamento entre clientes, pedidos, vendas e análises estratégicas.

RF02 - Cadastro de Produtos

O sistema deve permitir o cadastro de produtos contendo, no mínimo:

- ID
- Nome
- Preço
- Quantidade em estoque
- Categoria

O sistema deve validar obrigatoriamente que o preço do produto seja positivo e que a quantidade em estoque não seja negativa. A associação com categoria é obrigatória , permitindo que um produto esteja vinculado a uma categoria, conforme a regra adotada pelo sistema.

Exemplos de validação:

- **Preço válido:** 99.90 | **Preço inválido:** -10.00
- **Estoque válido:** 5 | **Estoque inválido:** -1

RF03 - Criação de Vendas

O sistema deve permitir a criação de venda vinculados a um cliente previamente cadastrado. Cada venda deve conter uma ou mais unidades de produtos, podendo incluir diferentes itens em uma mesma compra.

Uma venda deve possuir, no mínimo:

- ID da venda
- Cliente associado
- Produtos comprados
- Quantidade de cada produto
- Valor total
- Data da venda
- Status do venda

O sistema deve garantir que não seja criada uma venda sem cliente associado ou sem pelo menos um produto.

RF04 - Módulo de Visualização de Dados

O sistema deve possuir uma interface para visualização de dados comerciais e cadastrais por meio de gráficos, indicadores e painéis analíticos. Esse módulo deve exibir informações relacionadas a vendas, clientes e produtos.

Exemplos de indicadores:

- Quantidade total de vendas
- Valor total vendido
- Top clientes por valor comprado
- Top clientes por quantidade de pedidos
- Produto mais vendido
- Produto com maior valor de venda
- Vendas por estado, país e cidade
- Evolução das vendas ao longo do tempo

Esse módulo tem como objetivo facilitar o acompanhamento do desempenho comercial do sistema.

RF05 - Módulo de Auxílio à Decisão Estratégica

O sistema deve possuir uma interface voltada ao apoio à tomada de decisão estratégica. Esse módulo deve exibir informações relacionadas à classificação dos clientes, indicando o score de compra, a probabilidade de churn e o nível de risco de cancelamento ou abandono.

As informações devem permitir identificar clientes com maior valor para o negócio e clientes que apresentam risco elevado de deixar de comprar.

Exemplos de informações exibidas:

- Cliente
- Score de compra RFM
- Probabilidade de churn
- Classificação de risco
- Recência da última compra
- Frequência total de compras
- Valor total comprado

RF06 - Classificação e Ranking de Clientes

O sistema deve implementar um módulo de classificação e ranking de clientes com base em técnicas de Machine Learning e análise RFM.

O modelo utilizado para classificação de churn deve ser o **Random Forest**. Esse modelo deve ser usado para identificar:

- Clientes com maior propensão à compra
- Clientes com maior risco de churn

Além do modelo preditivo, o sistema deve calcular um score analítico de compra baseado em RFM, considerando recência, frequência, valor monetário e tempo de relacionamento. A classificação de risco de churn deve ser organizada em categorias, como: **Baixo, Médio, Alto e Crítico**.

RF07 - Consultas e Relatórios

O sistema deve permitir a consulta e listagem de clientes, produtos e pedidos, exibindo as informações relevantes de cada entidade.

As consultas devem permitir visualizar, no mínimo:

- Clientes e produtos cadastrados
- Vendas realizadas e itens de cada venda
- Histórico de compras por cliente e status das vendas

Além das consultas básicas, o sistema deve implementar relatórios gerenciais e relatórios estratégicos baseados nos dados cadastrados e analisados.

Relatórios gerenciais mínimos:

- Relatório de vendas por período
- Relatório de produtos mais vendidos

Relatórios estratégicos mínimos:

- Relatório de clientes com maior risco de churn
- Relatório de ranking de clientes por score de compra

Esses relatórios devem apoiar tanto a gestão operacional quanto a tomada de decisão estratégica.

2. Requisitos de Qualidade

O projeto deve demonstrar qualidade técnica tanto na organização do código quanto nos processos de desenvolvimento, validação, versionamento e escolha das soluções utilizadas.

RQ01 - Estrutura e Organização do Projeto

O projeto deve apresentar uma estrutura organizada e modular, com separação clara entre as responsabilidades de cada camada do sistema.

A aplicação deve evidenciar:

- Modularização do código
- Organização adequada de pastas
- Separação entre frontend e backend
- Separação entre regras de negócio, acesso a dados e interface
- Tipagem adequada dos dados
- Reutilização de componentes e funções

A estrutura deve facilitar manutenção, leitura, evolução e testes do sistema. Cada módulo deve possuir uma responsabilidade bem definida, evitando concentração excessiva de lógica em um único arquivo ou camada.

RQ02 - Boas Práticas de Codificação e Tratamento de Erros

O sistema deve seguir boas práticas de codificação, priorizando clareza, legibilidade, reaproveitamento de código e tratamento adequado de erros. Todos os dados recebidos pelo sistema devem ser validados conforme a necessidade de cada campo.

Exemplos de validações esperadas:

- E-mail do cliente obrigatório e em formato válido
- Preço do produto maior que zero
- Estoque do produto maior ou igual a zero
- Pedido obrigatoriamente vinculado a um cliente
- Pedido contendo pelo menos um produto
- Quantidade de produto maior que zero

O sistema também deve tratar falhas de execução de forma controlada, evitando erros não tratados que prejudiquem a experiência do usuário ou comprometam a integridade dos dados.

Devem ser considerados tratamentos para:

- Erro de conexão com banco de dados
- Dados inválidos enviados pelo usuário

- Tentativa de cadastro duplicado
- Produto ou cliente inexistente em um pedido
- Falhas durante processamento de relatórios ou análises

No módulo de Machine Learning, por exemplo, o código já utiliza blocos try/except para lidar com falhas na criação da tabela, extração de dados e gravação dos resultados, registrando os erros em logs.

RQ03 - Engenharia de Software e Versionamento

O projeto deve utilizar Git como ferramenta de controle de versão, evidenciando o processo de desenvolvimento ao longo do tempo.

O repositório deve apresentar commits:

- Atômicos e frequentes
- Com mensagens claras e semânticas
- Relacionados a pequenas entregas do projeto

Exemplos de mensagens adequadas:

- feat: adiciona cadastro de clientes
- feat: implementa pipeline de churn RFM
- fix: corrige validação de estoque negativo
- docs: adiciona documentação da arquitetura de dados
- refactor: separa regras de pedido em service

RQ04 - Seleção do Modelo e Dados para Treinamento

A escolha do modelo de Machine Learning deve ser compatível com o tamanho da aplicação, a quantidade de dados disponível e o poder computacional necessário para o processamento.

No projeto, o modelo selecionado para previsão de churn é o RandomForestClassifier, uma escolha adequada para problemas de classificação tabular. A seleção do modelo deve considerar:

- Volume de dados disponível
- Tempo necessário para treinamento
- Custo computacional
- Facilidade de interpretação dos resultados
- Capacidade de lidar com relações não lineares
- Robustez em bases com diferentes perfis de clientes

O projeto adota uma estratégia híbrida, combinando a previsão do Random Forest com uma regra de negócio baseada na recência da última compra. Essa decisão melhora a estabilidade do resultado e torna a classificação mais alinhada ao contexto do negócio.

RQ05 - Qualidade dos Dados

Além da escolha do modelo, o sistema deve garantir qualidade mínima dos dados utilizados nas análises e no treinamento.

O pipeline deve tratar:

- Registros duplicados
- Valores nulos
- Valores negativos incompatíveis com as métricas
- Outliers estatísticos
- Clientes sem histórico de compra

Essa etapa é essencial para evitar que inconsistências prejudiquem o cálculo do score RFM, a previsão de churn e os relatórios estratégicos. No módulo de ML, o pipeline de qualidade de dados remove duplicidades por cliente, preenche valores ausentes, corrige valores negativos e remove outliers utilizando Z-Score.

RQ06 - Observabilidade e Logs

O sistema deve registrar informações relevantes sobre sua execução, especialmente em processos de backend, BI e Machine Learning.

Os logs devem permitir acompanhar:

- Início e fim da execução do pipeline
- Quantidade de clientes processados, duplicidades removidas e outliers tratados
- Métricas de avaliação do modelo
- Erros de conexão, extração ou gravação

Essa prática facilita manutenção, depuração e acompanhamento da qualidade do processamento.

3. Requisitos Técnicos

O projeto deve atender a requisitos técnicos relacionados à interface, persistência de dados, uso de modelos de inteligência artificial, tratamento dos dados e desempenho na geração de relatórios.

RT01 - Interface Responsiva

O sistema deve possuir uma interface responsiva, capaz de se adaptar a diferentes tamanhos de tela e dispositivos.

A interface deve funcionar adequadamente em:

- Desktop
- Notebook
- Tablet
- Dispositivos móveis

Os módulos de cadastro, listagem, dashboards e relatórios devem manter boa usabilidade, organização visual e legibilidade mesmo em telas menores.

RT02 - Modelos de IA Sem Uso de APIs de LLMs

O sistema deve utilizar modelos de IA tradicionais ou algoritmos de Machine Learning locais, sendo proibido o uso de APIs de LLMs. No projeto, o modelo utilizado é o RandomForestClassifier, da biblioteca scikit-learn.

```
from sklearn.ensemble import RandomForestClassifier
```

```
model = RandomForestClassifier(  
    n_estimators=150,  
    max_depth=10,  
    random_state=42,  
    class_weight='balanced'  
)  
model.fit(X_train, y_train)
```

Essa abordagem atende ao requisito, pois não depende de APIs externas de modelos generativos ou LLMs.

RT03 - Bom Tratamento dos Dados para Random Forest

Para garantir a qualidade do modelo de Random Forest, o sistema deve realizar tratamento adequado dos dados antes do treinamento. No projeto, esse tratamento ocorre no método `data_quality_pipeline()`.

As principais etapas e códigos de exemplo são:

- **Remoção de duplicidades:**
`df = df.drop_duplicates(subset=['cliente_id'])`
- **Tratamento de nulos:**
`df = df.fillna(0)`
- **Correção de valores negativos:**
`for col in self.features:`
`inconsistent_mask = df[col] < 0`
`df.loc[inconsistent_mask, col] = 0`
- **Tratamento de outliers (Z-Score):**
`z_scores = np.abs(stats.zscore(df_compradores[['valor_total', 'frequencia_total']]))`
`outliers_mask = (z_scores > 3).any(axis=1)`
- **Normalização (MinMaxScaler):**
`scaler = MinMaxScaler()`
`X_scaled = scaler.fit_transform(X)`

RT04 - Geração de Relatórios em Tempo Aceitável

O sistema deve gerar relatórios gerenciais e relatórios de tomada de decisão em tempo aceitável, sem ultrapassar o limite máximo de 2 horas.

Para atender esse requisito de forma otimizada, os dados processados pelo pipeline de ML são previamente salvos na tabela `ml_cliente_scores`.

```
self.save_results(df)
```

Assim, os relatórios estratégicos não precisam treinar o modelo novamente a cada consulta; eles podem apenas consumir os resultados já calculados. Isso reduz drasticamente o tempo de resposta dos dashboards e relatórios, tornando o consumo dos dados muito mais eficiente.